

Single Cell RNA sequencing data analysis Using scanpy Pipeline

PREREQUISITES

```
# uv is faster than pip
!pip install uv
```

```
Requirement already satisfied: uv in c:\users\sanka\appdata\local\
programs\python\python314\lib\site-packages (0.9.18)
```

```
[notice] A new release of pip is available: 25.2 -> 25.3
[notice] To update, run: python.exe -m pip install --upgrade pip
```

```
# Creating a venv
!uv venv scrnaseq
```

```
Using CPython 3.14.0 interpreter at: C:\Users\sanka\AppData\Local\
Programs\Python\Python314\python.exe
Creating virtual environment at: scrnaseq
Activate with: scrnaseq\Scripts\activate
```

```
# Installing Jupyter ipykernel for Registering venv as a kernel
!uv pip install jupyter ipykernel
```

```
Using Python 3.14.0 environment at: scrnaseq
Resolved 97 packages in 1.84s
Prepared 84 packages in 12.00s
Installed 84 packages in 2.22s
```

```
+ anyio==4.12.0
+ argon2-cffi==25.1.0
+ argon2-cffi-bindings==25.1.0
+ arrow==1.4.0
+ asttokens==3.0.1
+ async-lru==2.0.5
+ babel==2.17.0
+ beautifulsoup4==4.14.3
+ bleach==6.3.0
+ certifi==2025.11.12
+ cffi==2.0.0
+ charset-normalizer==3.4.4
+ comm==0.2.3
+ debugpy==1.8.19
+ decorator==5.2.1
+ defusedxml==0.7.1
+ executing==2.2.1
+ fastjsonschema==2.21.2
+ fqdn==1.5.1
```

```
+ h11==0.16.0
+ httpcore==1.0.9
+ httpx==0.28.1
+ ipykernel==7.1.0
+ ipython==9.8.0
+ ipython-pygments-lexers==1.1.1
+ ipywidgets==8.1.8
+ isoduration==20.11.0
+ jedi==0.19.2
+ json5==0.12.1
+ jsonpointer==3.0.0
+ jsonschema==4.25.1
+ jsonschema-specifications==2025.9.1
+ jupyter==1.1.1
+ jupyter-client==8.7.0
+ jupyter-console==6.6.3
+ jupyter-core==5.9.1
+ jupyter-events==0.12.0
+ jupyter-lsp==2.3.0
+ jupyter-server==2.17.0
+ jupyter-server-terminals==0.5.3
+ jupyterlab==4.5.1
+ jupyterlab-pygments==0.3.0
+ jupyterlab-server==2.28.0
+ jupyterlab-widgets==3.0.16
+ lark==1.3.1
+ matplotlib-inline==0.2.1
+ mistune==3.1.4
+ nbclient==0.10.2
+ nbconvert==7.16.6
+ nbformat==5.10.4
+ nest-asyncio==1.6.0
+ notebook==7.5.1
+ notebook-shim==0.2.4
+ pandocfilters==1.5.1
+ parso==0.8.5
+ platformdirs==4.5.1
+ prometheus-client==0.23.1
+ prompt-toolkit==3.0.52
+ psutil==7.1.3
+ pure-eval==0.2.3
+ pycparser==2.23
+ python-json-logger==4.0.0
+ pywinpty==3.0.2
+ pyzmq==27.1.0
+ referencing==0.37.0
+ requests==2.32.5
+ rfc3339-validator==0.1.4
+ rfc3986-validator==0.1.1
```

```
+ rfc3987-syntax==1.1.0
+ rpds-py==0.30.0
+ send2trash==1.8.3
+ soupsieve==2.8
+ stack-data==0.6.3
+ terminado==0.18.1
+ tinycss2==1.4.0
+ tornado==6.5.4
+ traitlets==5.14.3
+ uri-template==1.3.0
+ urllib3==2.6.2
+ wcwidth==0.2.14
+ webcolors==25.10.0
+ webencodings==0.5.1
+ websocket-client==1.9.0
+ widgetsnbextension==4.0.15
```

Registering the venv as a kernel

```
!python -m ipykernel install --user --name=scrnaseq --display-
name="scrnaseq"
```

Installed kernelspec scrnaseq in C:\Users\sanka\AppData\Roaming\jupyter\kernels\scrnaseq

Install Packages / Libraries

```
!uv pip install pandas numpy scanpy scvi-tools
```

Using Python 3.14.0 environment at: .
Audited 4 packages in 1.46s

```
!uv pip list
```

Package	Version
absl-py	2.3.1
aiohappyeyeballs	2.6.1
aiohttp	3.13.2
aiosignal	1.4.0
anndata	0.12.7
anyio	4.12.0
argon2-cffi	25.1.0
argon2-cffi-bindings	25.1.0
array-api-compat	1.12.0
arrow	1.4.0
asttokens	3.0.1
async-lru	2.0.5
attrs	25.4.0
babel	2.17.0
beautifulsoup4	4.14.3
bleach	6.3.0

certifi	2025.11.12
cffi	2.0.0
charset-normalizer	3.4.4
colorama	0.4.6
comm	0.2.3
contourpy	1.3.3
cycler	0.12.1
debugpy	1.8.19
decorator	5.2.1
defusedxml	0.7.1
docrep	0.3.2
donfig	0.8.1.post1
executing	2.2.1
fastjsonschema	2.21.2
filelock	3.20.1
fonttools	4.61.1
fqdn	1.5.1
frozenlist	1.8.0
fsspec	2025.12.0
google-crc32c	1.8.0
grpcio	1.76.0
h11	0.16.0
h5py	3.15.1
httpcore	1.0.9
httpx	0.28.1
idna	3.11
ipykernel	7.1.0
ipython	9.8.0
ipython-pygments-lexers	1.1.1
ipywidgets	8.1.8
isoduration	20.11.0
jedi	0.19.2
jinja2	3.1.6
joblib	1.5.3
json5	0.12.1
jsonpointer	3.0.0
jsonschema	4.25.1
jsonschema-specifications	2025.9.1
jupyter	1.1.1
jupyter-client	8.7.0
jupyter-console	6.6.3
jupyter-core	5.9.1
jupyter-events	0.12.0
jupyter-lsp	2.3.0
jupyter-server	2.17.0
jupyter-server-terminals	0.5.3
jupyterlab	4.5.1
jupyterlab-pygments	0.3.0
jupyterlab-server	2.28.0

jupyterlab-widgets	3.0.16
kiwisolver	1.4.9
lark	1.3.1
legacy-api-wrap	1.5
lightning	2.6.0
lightning-utilities	0.15.2
llvmlite	0.46.0
markdown	3.10
markdown-it-py	4.0.0
markupsafe	3.0.3
matplotlib	3.10.8
matplotlib-inline	0.2.1
mdurl	0.1.2
mistune	3.1.4
ml-collections	1.1.0
mpmath	1.3.0
mudata	0.3.2
multidict	6.7.0
natsort	8.4.0
nbclient	0.10.2
nbconvert	7.16.6
nbformat	5.10.4
nest-asyncio	1.6.0
networkx	3.6.1
notebook	7.5.1
notebook-shim	0.2.4
numba	0.63.1
numcodecs	0.16.5
numpy	2.3.5
opt-einsum	3.4.0
packaging	25.0
pandas	2.3.3
pandocfilters	1.5.1
parso	0.8.5
patsy	1.0.2
pillow	12.0.0
platformdirs	4.5.1
prometheus-client	0.23.1
prompt-toolkit	3.0.52
propcache	0.4.1
protobuf	6.33.2
psutil	7.1.3
pure-eval	0.2.3
pycparser	2.23
pygments	2.19.2
pynndescent	0.5.13
pyarsing	3.2.5
pyro-api	0.1.2
pyro-ppl	1.9.1

python-dateutil	2.9.0.post0
python-json-logger	4.0.0
pytorch-lightning	2.6.0
pytz	2025.2
pywinpty	3.0.2
pyyaml	6.0.3
pyzmq	27.1.0
referencing	0.37.0
requests	2.32.5
rfc3339-validator	0.1.4
rfc3986-validator	0.1.1
rfc3987-syntax	1.1.0
rich	14.2.0
rpds-py	0.30.0
scanpy	1.11.5
scikit-learn	1.8.0
scikit-misc	0.5.2
scipy	1.16.3
scvi-tools	1.4.1
seaborn	0.13.2
send2trash	1.8.3
session-info2	0.2.3
setuptools	80.9.0
six	1.17.0
soupsieve	2.8
sparse	0.17.0
stack-data	0.6.3
statsmodels	0.14.6
sympy	1.14.0
tensorboard	2.20.0
tensorboard-data-server	0.7.2
terminado	0.18.1
threadpoolctl	3.6.0
tinycss2	1.4.0
torch	2.9.1
torchmetrics	1.8.2
tornado	6.5.4
tqdm	4.67.1
traitlets	5.14.3
typing-extensions	4.15.0
tzdata	2025.3
umap-learn	0.5.9.post2
uri-template	1.3.0
urllib3	2.6.2
wcwidth	0.2.14
webcolors	25.10.0
webencodings	0.5.1
websocket-client	1.9.0
werkzeug	3.1.4

```
widetsnbextension      4.0.15
xarray                 2025.12.0
yarl                   1.22.0
zarr                   3.1.5
```

```
Using Python 3.14.0 environment at: .
```

LOAD DATA

```
# Load packages

import scanpy as sc
import pandas as pd
import numpy as np
import scvi
import matplotlib.pyplot as plt

# adata =
sc.read_10x_mtx/sc.read_10x_h5/sc.read_h5ad("path/to/the/files")
# reads can be in different format

# loading the filtered data
adata = sc.read_10x_mtx("C:\\Users\\sanka\\Desktop\\scrnaseq\\data\\
pbmc3k_filtered_gene_bc_matrices\\filtered_gene_bc_matrices\\hgl9\\",
                      var_names = 'gene_symbols', cache = True)

# Using read_mtx would result into var_names(gene_names) and
obs_names(cell_barcode) to not load
# Instead of using \\ i could've used a "r-string" raw string which
doesn't considers \ as an escape character

# Make the var(gene) name unique
adata.var_names_make_unique()

# Checking whether the names are unique now or not
adata.var_names.is_unique

True
```

EDA

```
adata

AnnData object with n_obs × n_vars = 2700 × 32738
  var: 'gene_ids'

# INITIAL EDA

print(f"Total cells : {len(adata.obs)}" )
print(f"Total Genes : {len(adata.var)}" )
```

```
# print(f"sparsity = {1 - sum(adata.X > 0) / len(adata.obs) *
len(adata.var)} " )
### NotImplementedError: subtracting a sparse array from a nonzero
scalar is not supported ###
```

```
Total cells : 2700
Total Genes : 32738
```

```
total_elements = adata.X.shape[0] * adata.X.shape[1]
nonzero_elements = adata.X.nnz
zero_elements = total_elements - nonzero_elements
sparsity = zero_elements / total_elements
```

```
print(f"Sparsity : {(sparsity*100):.2f}%")
```

```
Sparsity : 97.41%
```

```
# UMI Distribution
```

```
umi_counts = adata.X.sum(axis=0).A1 # .A1 converts the result to a 1D
NumPy array (single entries ).
print(f"Median UMI per cell : {np.median(umi_counts):.2f}")
print(f"Cells with >500 counts : {(umi_counts > 500).sum()}")
```

```
Median UMI per cell : 1.00
Cells with >500 counts : 1355
```

```
# Calculating the qc metrics of adata
```

```
sc.pp.calculate_qc_metrics(adata,percent_top=None,log1p=False,inplace=
True)
```

```
adata
```

```
AnnData object with n_obs × n_vars = 2700 × 32738
```

```
  obs: 'n_genes_by_counts', 'total_counts'
  var: 'gene_ids', 'n_cells_by_counts', 'mean_counts',
  'pct_dropout_by_counts', 'total_counts'
```

```
adata.obs
```

	n_genes_by_counts	total_counts
AAACATACAACCAC-1	781	2421.0
AAACATTGAGCTAC-1	1352	4903.0
AAACATTGATCAGC-1	1131	3149.0
AAACCGTGCTTCCG-1	960	2639.0
AAACCGTGTATGCG-1	522	981.0
...	...	...
TTTCGAACTCTCAT-1	1155	3461.0
TTTCTACTGAGGCA-1	1227	3447.0
TTTCTACTTCCTCG-1	622	1684.0
TTTGCATGAGAGGC-1	454	1024.0
TTTGCATGCCTCAC-1	724	1985.0


```
[2700 rows x 2 columns]
```

Preprocessing AND Quality Control

```
# Getting How many Genes are there that are expressed in less than 3
cells
print(f"Min Number of cells a gene is expressed  :
{min(adata.var["n_cells_by_counts"])} ")
print(f"Max Number of cells a gene is expressed :
{max(adata.var["n_cells_by_counts"])} ")
Gene_exp = adata.var_names[(adata.var["n_cells_by_counts"] <
3 )].tolist()
print(f"Genes Expressed in less than 3 cells : {len(Gene_exp)} \n ")

# Getting How many Cells have less than 100 genes expressed
print(f"Min Number of genes in a cell :
{min(adata.obs["n_genes_by_counts"])} ")
print(f"Max Number of genes in a cell :
{max(adata.obs["n_genes_by_counts"])} ")
Per_cell_gene_exp = adata.obs_names[(adata.obs["n_genes_by_counts"]<
200 )].tolist()
print(f"Number of cells with less than 200 genes expressed
( Insignificant ) : {len(Per_cell_gene_exp)}")

Min Number of cells a gene is expressed  : 0
Max Number of cells a gene is expressed : 2700
Genes Expressed in less than 3 cells : 19024

Min Number of genes in a cell : 212
Max Number of genes in a cell : 3422
Number of cells with less than 200 genes expressed ( Insignificant ) :
0

# BASIC FILTERING
sc.pp.filter_genes(adata, min_cells = 3 )
print(f"Removed {len(Gene_exp)} Genes")

sc.pp.filter_cells(adata, min_genes = 200 )
print(f"Removed {len(Per_cell_gene_exp)} cells ")

Gene_exp = adata.var_names[(adata.var["n_cells_by_counts"] <
3)].tolist()
Per_cell_gene_exp = adata.obs_names[(adata.obs["n_genes_by_counts"]<
```

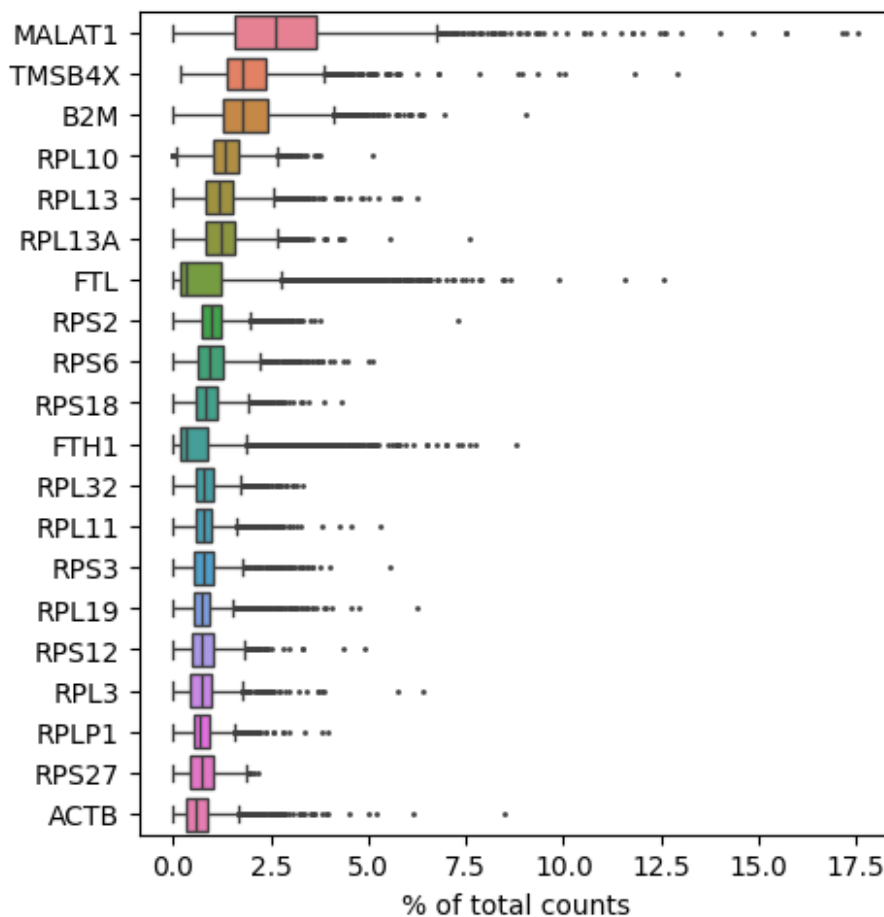
```
200 )].tolist()
# print(f"Genes Expressed in less than 3 cells : {len(Gene_exp)} ")
# print(f"Cells with less than 200 gene expression :
{len(Per_cell_gene_exp)} ")
```

Removed 19024 Genes
Removed 0 cells

```
print(f"Total cells after basic filtering : {len(adata.obs)}" )
print(f"Total Genes after basic filtering : {len(adata.var)}" )
```

Total cells after basic filtering : 2700
Total Genes after basic filtering : 13714

```
# Plotting the top 20 highly expressed genes
sc.pl.highest_expr_genes(adata, n_top=20)
```



```
adata_qc = adata.copy()
```

```
# Identify gene types
```

```
adata_qc.var['mt'] = adata_qc.var_names.str.startswith('MT-')
```

```

adata_qc.var['ribo'] = adata_qc.var_names.str.startswith(('RPS',
'RPL'))
adata_qc.var['hemoglobin'] =
adata_qc.var_names.str.contains('^HB^(P)')

print(f"Found {adata_qc.var["mt"].sum()} Mitochondrial genes. " )
print(f"Found {adata_qc.var["ribo"].sum()} Ribosomal genes. " )
print(f"Found {adata_qc.var["hemoglobin"].sum()} Hemoglobin genes. " )

Found 13 Mitochondrial genes.
Found 99 Ribosomal genes.
Found 4 Hemoglobin genes.

# # plotting qc metrics

# # Set up the figure and axes objects for a 3x2 grid
# fig, axes = plt.subplots(nrows=3, ncols=2, figsize=(12, 18))
# # ----- This will create a 3,2 array of axes [[ax1, ax2], [ax3,
ax4], [ax5, ax6]] -----

# # sc.pl.hist(adata_qc, keys="n_genes_by_counts" , show=False , bins=
50 )
# ## ERRORR : module 'scanpy.plotting' has no attribute 'hist'

# # First plot : Plots of total counts of genes in each cell ( Genes
per cell )

# ax1_hist = axes[0,0]
# plt.hist(adata_qc.obs["n_genes_by_counts"] , bins = 50 , alpha =
0.7, color = "steelblue", edgecolor = 'black')
# plt.xlabel("Genes per cell")
# plt.ylabel("Number of cells ")
# plt.title("Gene per cell distribution")
#
plt.axvline(np.median(adata.obs["n_genes_by_counts"]),color="red",line
style = "--" , label = f"Median
{np.median(adata.obs["n_genes_by_counts"]):.0f} " )
# plt.legend()
# plt.show()

# # plt.figure(figsize = (10,6))
# plt.subplot(3,2,2)
# sc.pl.violin(adata_qc , "n_genes_by_counts" , jitter=0.7)
# # Second plot : Histogram plot of total Umi counts per cell

# # plt.figure(figsize = (10,6))
# plt.subplot(3,2,3)
# plt.hist(adata_qc.obs["total_counts"] , bins = 50 , alpha = 0.7,

```

```

color = "yellow", edgecolor = 'black')
# plt.xlabel("UMI Detected per cell")
# plt.ylabel("Number of cells ")
# plt.title("UMI distribution")
#
plt.axvline(np.median(adata.obs["total_counts"]),color="red",linestyle
= "--" , label = f"Median {np.median(adata.obs["total_counts"]):.0f} "
)
# plt.legend()
# plt.show()

# # plt.hist(adata_qc.obs["n_genes_by_counts"] , bins = 50 , alpha =
0.7, color = "steelblue", edgecolor = 'black')
# # plt.xlabel("Genes per cell")
# # plt.ylabel("Number of cells ")
# # plt.title("Gene per cell distribution")
# #
plt.axvline(np.median(adata.obs["n_genes_by_counts"]),color="red",line
style = "--" , label = f"Median
{np.median(adata.obs["n_genes_by_counts"]):.0f} " )
# # plt.legend()
# # plt.show()

# Calculating the QC Metrics Of MT, RIBO and HEMO CONTENT
sc.pp.calculate_qc_metrics(adata_qc, loglp=False,percent_top=None,
qc_vars=["mt","ribo","hemoglobin"], inplace=True)

# More Structured Through AI

# Plotting qc metrics

# Set up the figure and axes objects for a 3x2 grid
fig, axes = plt.subplots(nrows=3, ncols=2, figsize=(12, 18))
# axes is now a 2D numpy array: [[ax1, ax2], [ax3, ax4], [ax5, ax6]]

# --- Row 1: Genes per Cell ---
# 1a: Histogram
ax1_hist = axes[0, 0]
ax1_hist.hist(adata_qc.obs["n_genes_by_counts"], bins=50, alpha=0.7,
color="steelblue", edgecolor='black')
ax1_hist.set_xlabel("Genes per cell")
ax1_hist.set_ylabel("Number of cells")
ax1_hist.set_title("Gene distribution (Histogram)")
median_genes = np.median(adata.obs["n_genes_by_counts"])
ax1_hist.axvline(median_genes, color="red", linestyle="--",
label=f"Median {median_genes:.0f}")
ax1_hist.legend()

```

```

# 1b: Violin Plot
ax1_viol = axes[0, 1]
sc.pl.violin(adata_qc, "n_genes_by_counts", jitter=0.4, ax=ax1_viol,
show=False )
ax1_viol.set_title("Genes per cell (Violin Plot)")
ax1_viol.set_xlabel("") # Clear redundant x-label

# --- Row 2: UMI Counts per Cell ---
# 2a: Histogram
ax2_hist = axes[1, 0]
ax2_hist.hist(adata_qc.obs["total_counts"], bins=50, alpha=0.7,
color="yellow", edgecolor='black')
ax2_hist.set_xlabel("UMI Detected per cell")
ax2_hist.set_ylabel("Number of cells")
ax2_hist.set_title("UMI distribution (Histogram)")
median_counts = np.median(adata.obs["total_counts"])
ax2_hist.axvline(median_counts, color="red", linestyle="--",
label=f"Median {median_counts:.0f}")
ax2_hist.legend()

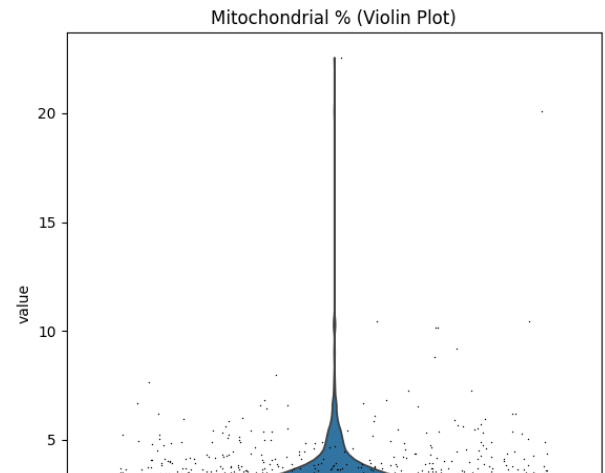
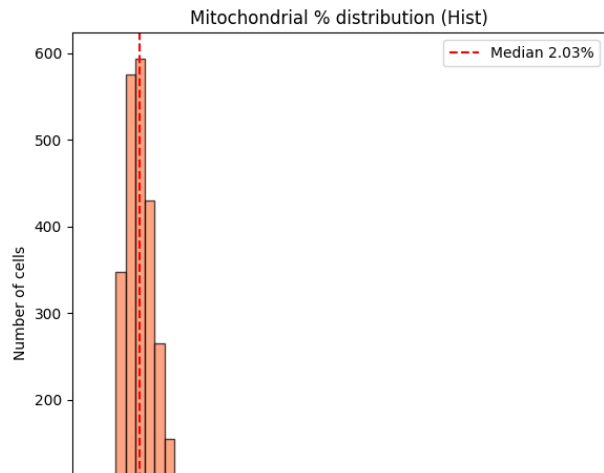
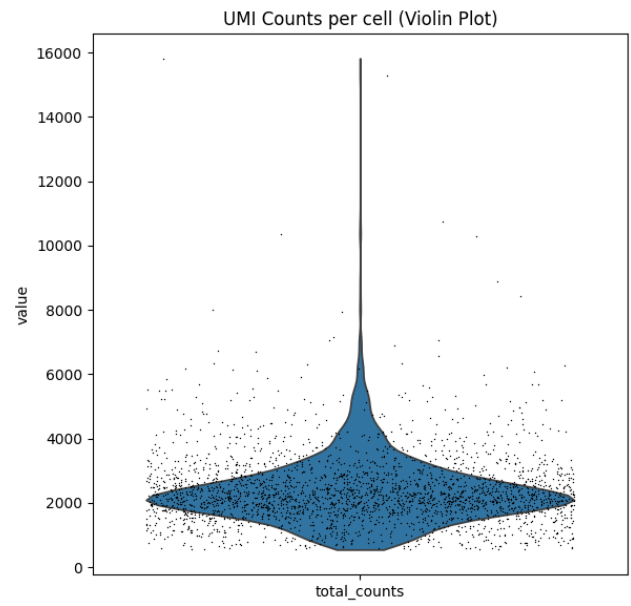
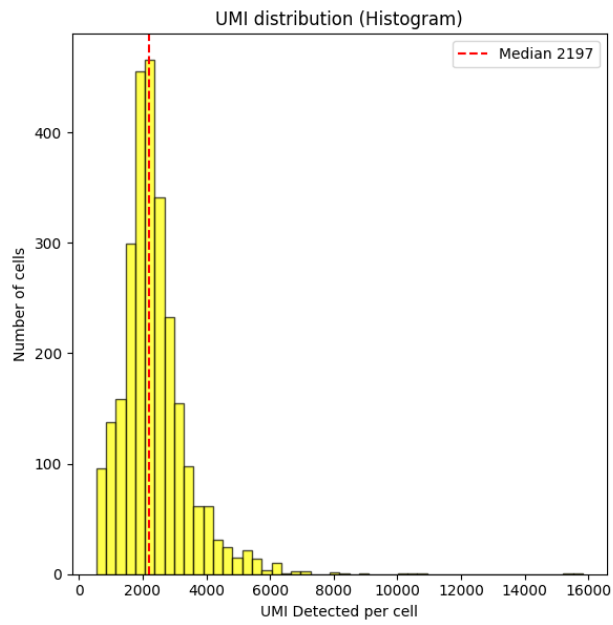
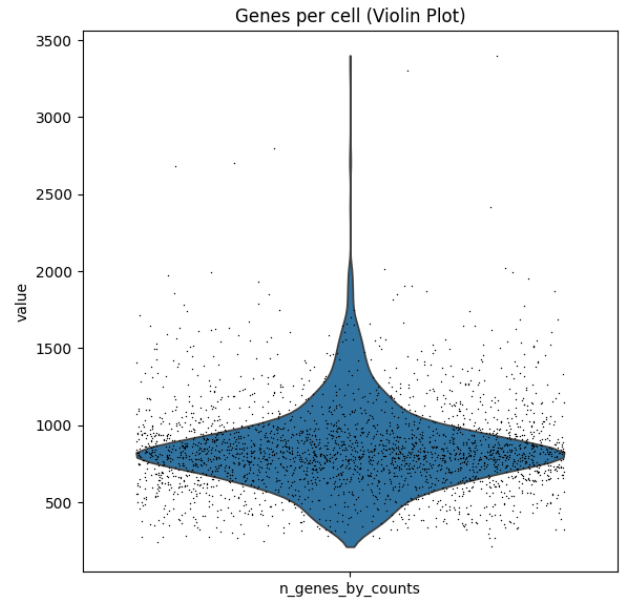
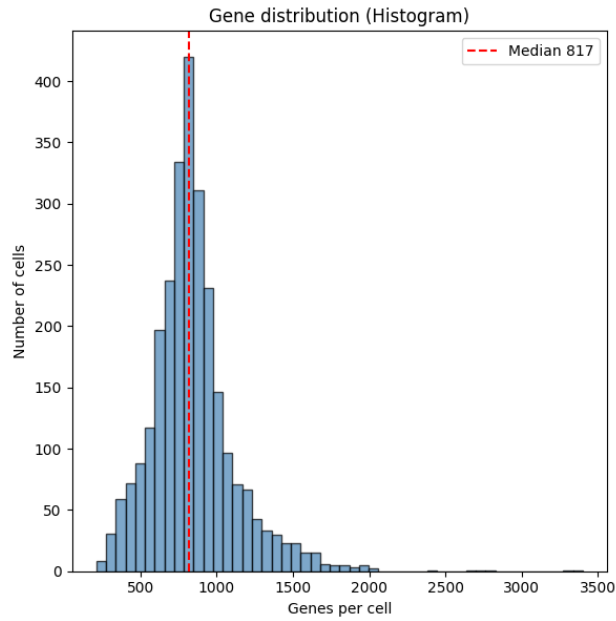
# 2b: Violin Plot
ax2_viol = axes[1, 1]
sc.pl.violin(adata_qc, "total_counts", jitter=0.4, ax=ax2_viol,
show=False)
ax2_viol.set_title("UMI Counts per cell (Violin Plot)")
ax2_viol.set_xlabel("")

# --- Row 3: Mitochondrial Distribution by Percentage---
# 3a: Histogram
ax3_hist = axes[2, 0]
ax3_hist.hist(adata_qc.obs["pct_counts_mt"], bins=50, alpha=0.7,
color="coral", edgecolor='black')
ax3_hist.set_xlabel("% MT counts per cell")
ax3_hist.set_ylabel("Number of cells")
ax3_hist.set_title("Mitochondrial % distribution (Hist)")
median_mt = np.median(adata_qc.obs["pct_counts_mt"])
ax3_hist.axvline(median_mt, color="red", linestyle="--",
label=f"Median {median_mt:.2f}%")
ax3_hist.legend()

# 3b: Violin Plot
ax3_viol = axes[2, 1]
sc.pl.violin(adata_qc, "pct_counts_mt", jitter=0.4, ax=ax3_viol,
show=False)
ax3_viol.set_title("Mitochondrial % (Violin Plot)")
ax3_viol.set_xlabel("")

```

```
# Adjust layout to prevent titles/labels from overlapping  
plt.tight_layout()  
plt.show()
```



```
# Filtering the high Mitochondrial content cells
```

```
high_mito = adata_qc.obs[(adata_qc.obs["pct_counts_mt"] > 20 )]
high_mito_index_list = high_mito.index.tolist()
high_mito
```

	n_genes_by_counts	total_counts	n_genes
total_counts_mt \			
CGTACCTGGACGAG-1	407	833.0	407
188.0			
CTCATTGATTGCTT-1	300	592.0	300
119.0			

	pct_counts_mt	total_counts_ribo	pct_counts_ribo \
CGTACCTGGACGAG-1	22.569027	35.0	4.201681
CTCATTGATTGCTT-1	20.101353	86.0	14.527027

	total_counts_hemoglobin	pct_counts_hemoglobin
CGTACCTGGACGAG-1	0.0	0.0
CTCATTGATTGCTT-1	0.0	0.0

```
# Some more Filtering based upon the Graphs
```

```
adata_qc = adata_qc[
    (adata_qc.obs.n_genes_by_counts < 2500) &
    (adata_qc.obs.n_genes_by_counts > 200) & (adata_qc.obs.pct_counts_mt <
5), :, ].copy()
adata_qc.layers["counts"] = adata_qc.X.copy()
```

```
adata_qc
```

```
AnnData object with n_obs × n_vars = 2638 × 13714
  obs: 'n_genes_by_counts', 'total_counts', 'n_genes',
'total_counts_mt', 'pct_counts_mt', 'total_counts_ribo',
'pct_counts_ribo', 'total_counts_hemoglobin', 'pct_counts_hemoglobin'
  var: 'gene_ids', 'n_cells_by_counts', 'mean_counts',
'pct_dropout_by_counts', 'total_counts', 'n_cells', 'mt', 'ribo',
'hemoglobin'
  layers: 'counts'
```

Doublet detection

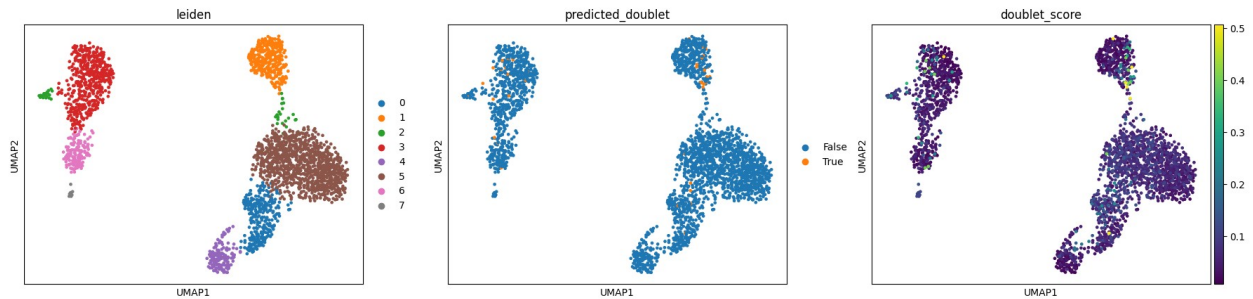
```
!uv pip install scikit-image
```

```
Using Python 3.14.0 environment at: .
Audited 1 package in 9ms
```

```
sc.pp.scrublet(adata_qc)
```



```
# AFTER CLUSTERING STEP
sc.pl.umap(
    adata_qc,
    color=["leiden", "predicted_doublet", "doublet_score"],
)
```

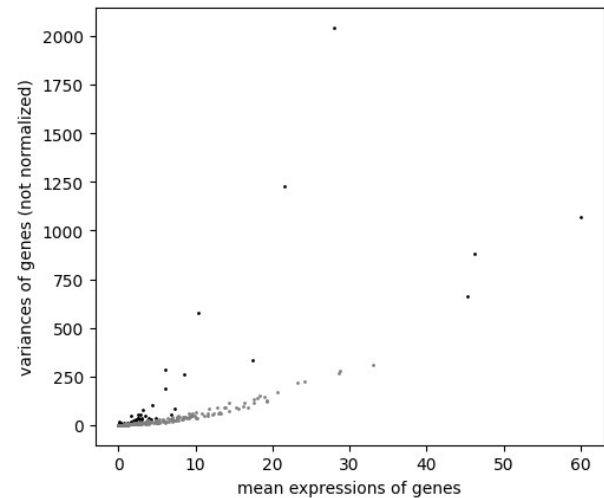
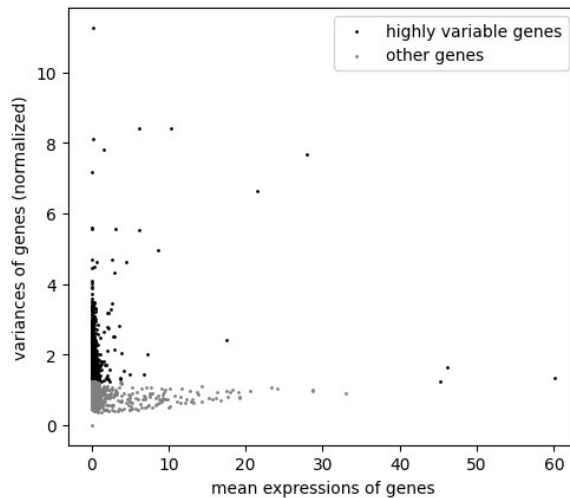


Normalization

```
sc.pp.normalize_total(adata_qc, target_sum=1e4)
# Log1p transformation
sc.pp.log1p(adata_qc)
```

Feature Selection

```
sc.pp.highly_variable_genes(
    adata_qc,
    n_top_genes=2000,
    min_mean=0.0125,
    max_mean=3,
    min_disp=0.5,
    flavor="seurat_v3",
    layer="counts",
    inplace=True
)
sc.pl.highly_variable_genes(adata_qc)
```



```
# Selecting only highly variable genes out of 13714 genes
adata_qc = adata_qc[:, adata_qc.var.highly_variable]
```

Dimensionality Reduction

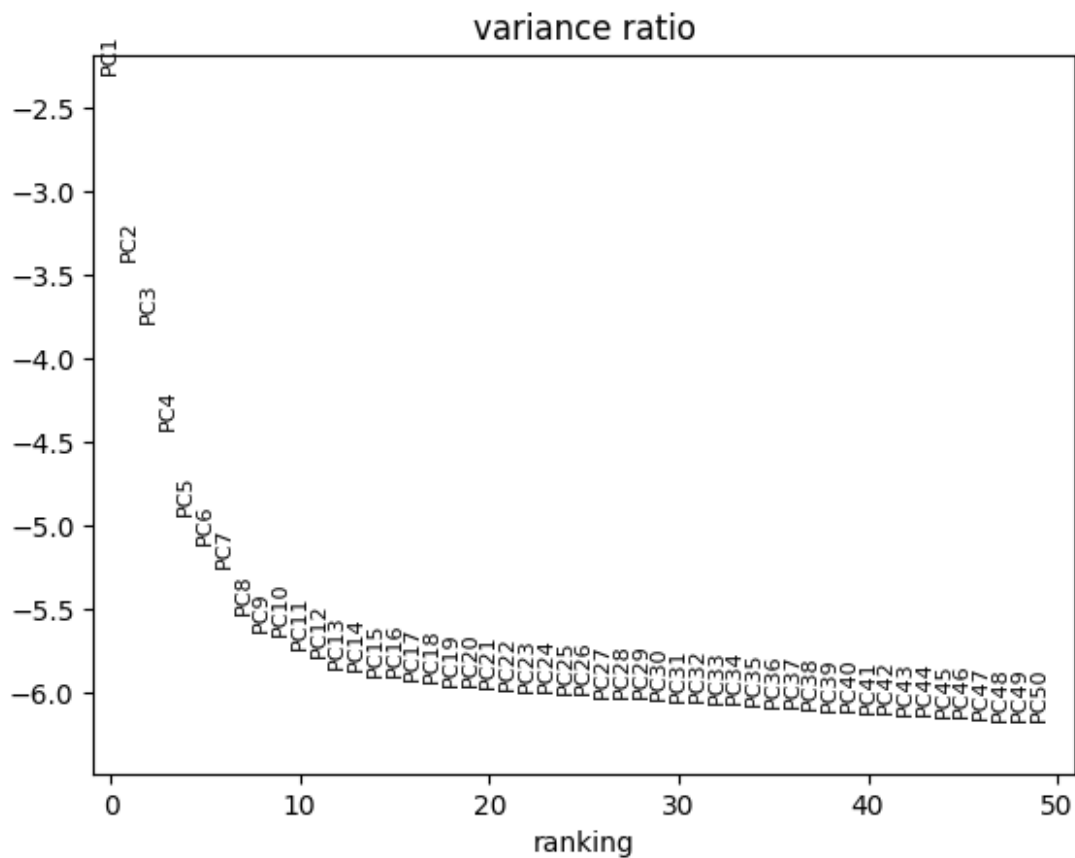
```
# Calculating PCs
sc.tl.pca(adata_qc)
```

```
# BY DEFAULT IT CALCULATES 50 PCs
# Results stored in adata_qc.obsm["X_pca"]
```

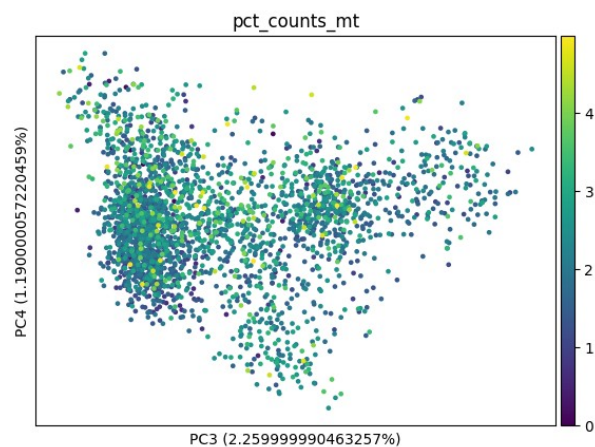
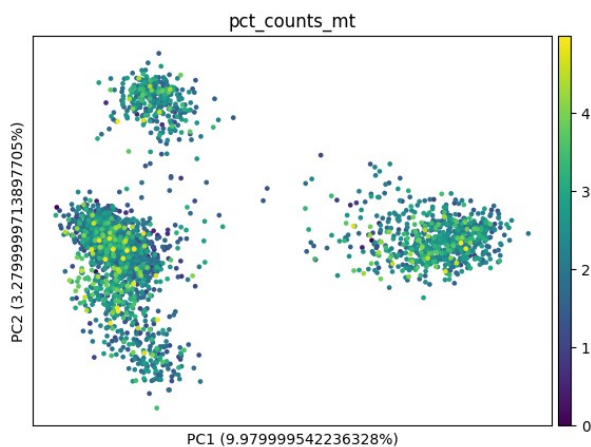
```
C:\Users\sanka\Desktop\scrnaseq\Lib\site-packages\scanpy\
preprocessing\_pca\__init__.py:384: ImplicitModificationWarning:
Setting element `obsm['X_pca']` of view, initializing view as actual.
adata.obsm[key_obsm] = x_pca
```

```
# Visualizing PCAs
```

```
sc.pl.pca_variance_ratio(adata_qc, n_pcs=50, log=True)
```



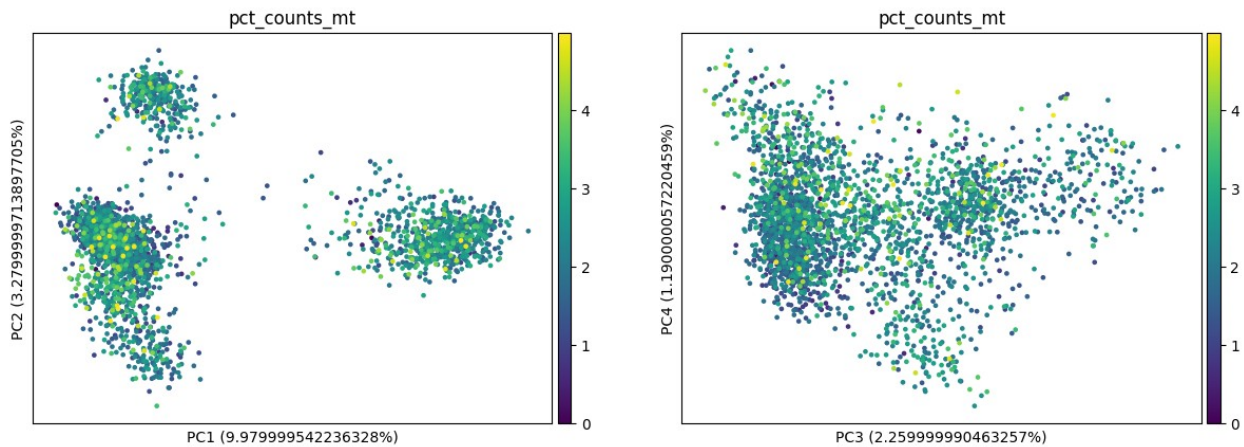
```
sc.pl.pca(
    adata_qc,
    color=[ "pct_counts_mt" ],
    dimensions=[ (0, 1), (2, 3) ],
    annotate_var_explained=True,
)
```



```
adata_qc.layers["scaled"] = adata_qc.X.toarray()
sc.pp.regress_out(adata_qc, ["total_counts", "pct_counts_mt"],
layer="scaled")
# Removes the linear effects of specified variables
# Total_counts and pct_counts_mt's data from scaled layer
sc.pp.scale(adata_qc, max_value=10, layer="scaled")
# This applies z-score normalization (scaling) to the data within the
"scaled" layer.

sc.tl.pca(adata_qc)

sc.pl.pca(
    adata_qc,
    color=["pct_counts_mt"],
    dimensions=[(0, 1), (2, 3)],
    annotate_var_explained=True,
)
```



```
# Checking which genes contributes to pca1 and pca2 the most and the
least
# Checking this Will tell how much which PC is contributing to the
biology
```

```
sc.pl.pca_loadings(adata_qc, components=(1, 2), include_lowest=True)
```


CLUSTERING

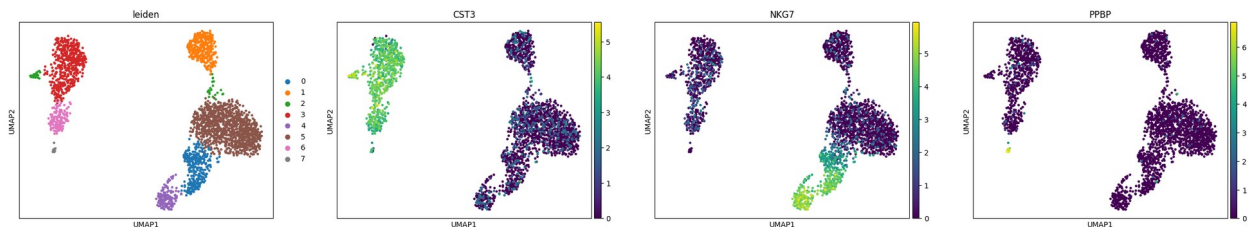
```
!uv pip install leidenalg
```

```
Using Python 3.14.0 environment at: .  
Audited 1 package in 7ms
```

```
# Clustering Using the Leiden Algorithm
```

```
sc.tl.leiden(adata_qc, resolution = 0.5, flavor = "igraph",  
n_iterations = 2)
```

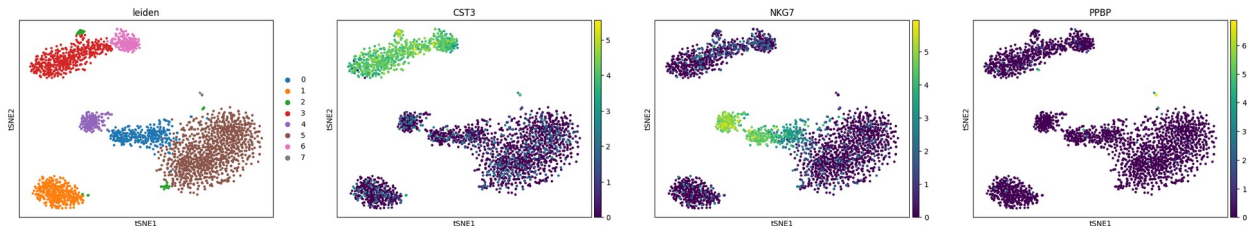
```
sc.pl.umap(adata_qc, color = ["leiden", "CST3", "NKG7", "PPBP"])
```



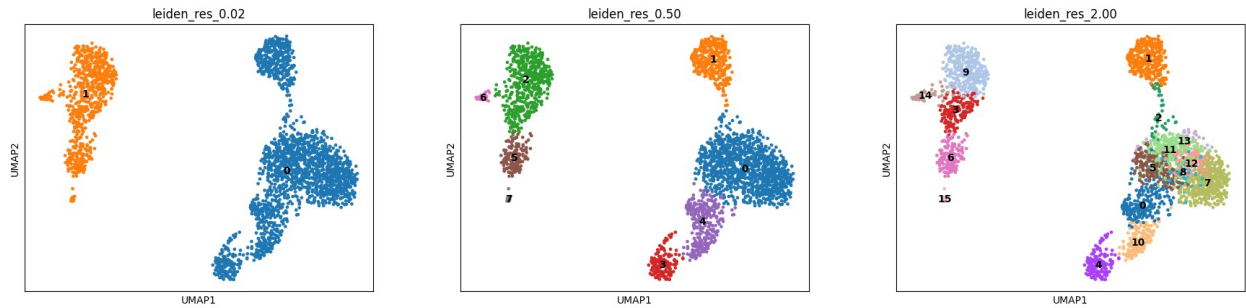
```
# t-sne Graphical Calculation
```

```
sc.tl.tsne(adata_qc, n_pcs=40)
```

```
sc.pl.tsne(adata_qc, color=["leiden", "CST3", "NKG7", "PPBP"])
```



```
for res in [0.02, 0.5, 2.0]:  
    sc.tl.leiden(adata_qc, key_added=f"leiden_res_{res:4.2f}",  
resolution=res, flavor="igraph")  
  
sc.pl.umap(  
    adata_qc,  
    color=["leiden_res_0.02", "leiden_res_0.50", "leiden_res_2.00"],  
    legend_loc="on data",  
)
```



CELL ANNOTATION - MARKER GENE IDENTIFICATION

This data will require to find the marker genes Manually , based on each gene's expression in a cell cluster , if in a cell cluster a gene is highly expressed in comparison to other clusters or cells then possibly it's a marker gene.

For Example : We'll test The GENE A's Expression in Two groups

Group 1 = cells of our desired cluster

Group 2 = All other cells

If the GENE A's expression in GROUP 1 is higher than GROUP 2 Then it'll be a marker gene of the Cells in Group 1

The same test is done for every gene and for every cluster.

Result: each cluster gets a ranked list of genes

Basically Top genes = markers

